Training: Passing tensors from tf to tfq using cirq to train

https://github.com/tensorflow/quantum/issues/321

ROW 132

Training: Passing tensors from tf to tfq using cirq to train #321

New issue

Closed

m0tela01 opened on Jul 24, 2020 · edited by m0tela01 — Edits

Hello

Is there a way to pass a tensor in a form that is usable by cirq when it is passed from an upstream layer during training?

I have a tensor from a Dense layer. I want to run it through a custom layer and perform some tensor operations. This layer then outputs/returns a cirq circuit I designed.
We can remove the functions in the layer or the layer entirely to make the problem more clear.
Is there a way to pass a tensor into a cirq circuit using tfq/tf/cirq so that it can be trained?

I was assuming that since tensorflow is graph based none of the values would be accessible for training (i.e. you cannot `get_weights()` or evaluate the network in the middle of training to produce a numpy array/float value to pass into the circuit. Is there any way to go about this from any of the packages inside of tfq/tf/cirq/keras? I have tried using an `tfq.layers.Expectation()` by passing the tensor as the initializer. I have tried to find a solution to this using cirq functions as well but have not had success. Was thinking of also posting this in the cirq issues but (at least right now) believe this is more related to tfq than cirq.

Please let me know if there is anything below that is unclear or needs further explanation. Thanks in advance.

Code

I have tried to input a tensor into cirq and gotten this:

```
place = tf.compat.v1.placeholder(tf.float32, shape=(4,1))
placeholder = cirq.ry(place)(cirq.GridQubit(0,0))
placeholder ## -> cirq.ry(np.pi*<tf.Tensor 'truediv_25:0' shape=(4, 1) dtype=float32>).on(cirq.GridQubit(0, 0))
```

When it is embedded inside of a circuit and called like this it gives an error:

```
circuit1 = cirq.Circuit()
circuit1.append(cirq.ry(place)(cirq.GridQubit(0,0)))
circuit1 ## ->TypeError: int() argument must be a string, a bytes-like object or a number, not 'Tensor'
```

When it is called inside of the layer using `tfq.convert_to_tensor ` the error shows:
`TypeError: can't pickle _thread.RLock objects`

The functions from the layer:
GetAngles3(weights): *creates tensors with shape=(4,)*
CreateCircuit2(x): *creates a cirq circuit using tfq.convert_to_tensor ` following the documetation*

Here is the layer itself:

```
class ASDF2(tf.keras.layers.Layer):
    def __init__(self, outshape=4, qubits=cirq.GridQubit.rect(1, 4)):
        super(ASDF2, self).__init__()
        self.outshape = outshape
        self.qubits = qubits

    def build(self, input_shape):
        self.kernel = self.add_weight(name='kernel',
                                      shape=(int(input_shape[1]), self.outshape),
                                      trainable=True)

        super(ASDF2, self).build(input_shape)


    def call(self, inputs, **kwargs):
        try:
            x = GetAngles3(inputs)
```

Assignees
MichaelBroughton

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

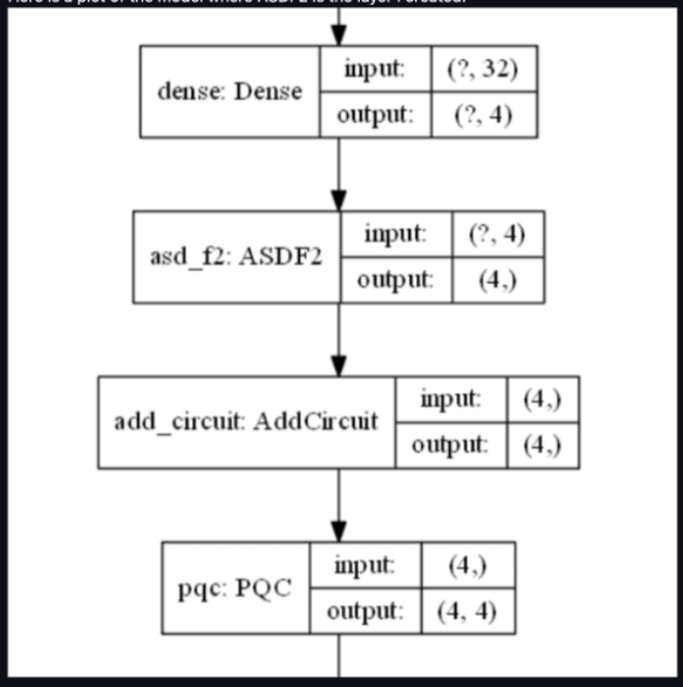
Participants

```
        try:
            x = GetAngles3(inputs)
            y = CreateCircuit2(x)
            return y
        except:
            x = tf.dtypes.cast(inputs, tf.dtypes.string)
            return tf.keras.backend.squeeze(x, 0)
```

Here is the output when the model runs:

```
Train on 78 samples, validate on 26 samples
Epoch 1/25
**********inputs
Tensor("model/dense/Sigmoid:0", shape=(None, 4), dtype=float32)
**********GetAngles
Tensor("model/dense/Sigmoid:0", shape=(None, 4), dtype=float32)
[<tf.Tensor 'model/asd_f2/Asin:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_1:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_2:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_3:0' shape=(None, 4) dtype=float32>]
**********createCircuit
WARNING:tensorflow:Gradients do not exist for variables ['conv1d/kernel:0', 'conv1d/bias:0', 'dense/kernel:0', 'dense/bias:0', 'asd_f2/kernel:0'] when minimizing the loss.
**********inputs
Tensor("model/dense/Sigmoid:0", shape=(None, 4), dtype=float32)
**********GetAngles
Tensor("model/dense/Sigmoid:0", shape=(None, 4), dtype=float32)
[<tf.Tensor 'model/asd_f2/Asin:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_1:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_2:0' shape=(None, 4) dtype=float32>, <tf.Tensor 'model/asd_f2/mul_3:0' shape=(None, 4) dtype=float32>]
**********createCircuit
WARNING:tensorflow:Gradients do not exist for variables ['conv1d/kernel:0', 'conv1d/bias:0', 'dense/kernel:0', 'dense/bias:0', 'asd_f2/kernel:0'] when minimizing the loss.
```

Here is a plot of the model where ASDF2 is the layer I created:



MichaelBroughton on Jul 26, 2020 — Collaborator

I'm having a hard time following exactly what's going on here. It looks like there might be several different issues at play. With regards to:

> I have tried to input a tensor into cirq and gotten this:
> place = tf.compat.v1.placeholder(tf.float32, shape=(4,1))
> placeholder = cirq.ry(place)(cirq.GridQubit(0,0))
> placeholder ## -> cirq.ry(np.pi*<tf.Tensor 'truediv_25:0' shape=(4, 1) dtype=float32>).on(cirq.GridQubit(0, 0))
> When it is embedded inside of a circuit and called like this it gives an error:
> circuit1 = cirq.Circuit()
> circuit1.append(cirq.ry(place)(cirq.GridQubit(0,0)))
> circuit1 ## ->TypeError: int() argument must be a string, a bytes-like object or a number, not 'Tensor'
> When it is called inside of the layer using `tfq.convert_to_tensor` the error shows:
> TypeError: can't pickle _thread.RLock objects

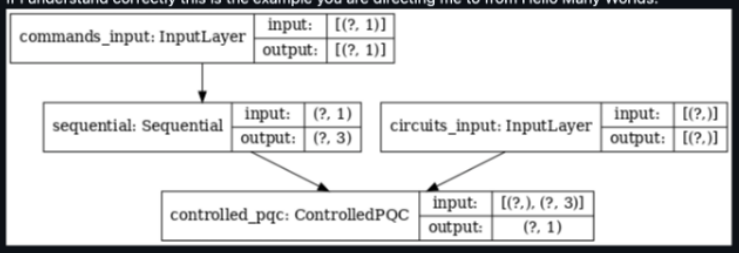
This is caused by misuse of TensorFlow and Cirq together. Cirq does not support using or operating on `tf.Tensor` objects (otherwise we wouldn't need TensorFlow Quantum). In order to understand how to create and evaluate parameterized circuits in TensorFlow Quantum I would recommend checking out: https://www.tensorflow.org/quantum/tutorials/hello_many_worlds

This tutorial showcases some examples of how upstream `tf.Tensor`s can be passed into the variables in quantum circuits that are downstream in the compute graph like you want to do with your `ASDF2` layer.

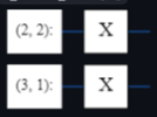
Does this help clear things up ?

m0tela01 on Jul 27, 2020 · edited by m0tela01 — Edits — Author

If I understand correctly this is the example you are directing me to from Hello Many Worlds:



This is close to what I am trying to do.
I am trying to compose my circuits with the data I have similar to the example from the MNIST Classification example where the circuits are the representative states of the data:
`x_train_circ[0]`



My main question is can I have a model that is DENSE > PQC > DENSE (where the PQC is created based on the output of the first DENSE)? If this cannot be done then is there a way to pass data (not a simple clustering state) to both the LHS and RHS in the first figure above? If I cannot do this with cirq is there another way to perform this with just TFQ/TF?

Thanks for the response. Missed the email yesterday.

m0tela01 on Jul 31, 2020 · edited by m0tela01 — Edits — Author

Any update on this? Is it feasible? Just to reiterate. @MichaelBroughton the response you gave is not a solution to my question.

MichaelBroughton on Jul 31, 2020 · edited by MichaelBroughton Edits Collaborator ···

Sorry for the slow response (I'm on vacation).

I'm having a hard time following exactly what you want in your follow up question. When you say:

> My main question is can I have a model that is DENSE > PQC > DENSE (where the PQC is created based on the output of the first DENSE)?

I'm pretty sure now you don't want something like this (Just in case this is what you want I've added code):

```
numerical_input = tf.keras.layers.Input(...)
circuit_input = tf.keras.layers.Input(...)
a = tf.layers.Dense(32)(numerical_input)
b = tfq.layers.ControlledPQC(model_circuit, some operator)(circuit_input, a)
c = tf.layers.Dense(32)(b)
.
.
.
out = ...
model = tf.keras.Model(inputs=[numerical_input, circuit_input], outputs=out)
model.([some tensor of numbers, some tensor of circuits])
```

Where Dense layer `a` controls the learnable parameters that go into the circuit which **models** the quantum datapoint circuits coming in from the `circuit_input` tensor.

What I'm guessing you do want is: Be able to influence the **quantum datapoint circuits** themselves before they get fed into some kind of PQC / downstream quantum model things. For this you need to make use of `tfq.layers.Expectation` which requires a bit more bookkeeping compared to PQC:

```
# Choose controllable symbols for datapoints and symbols to be learned by model.
datapoint_symbols = sympy.symbols('alpha beta')
model_symbols = sympy.Symbol('gamma')

# Create model circuit
# Note: Must contain only symbols from `model_symbols` and not `datapoint_symbols`
quantum_model_circuit = cirq.Circuit(...)

# Create datapoint circuits
# Note: All circuits must contain only symbols from `datapoint_symbols`
datapoint_circuits = [cirq.Circuit(...), cirq.Circuit(...), ...]

numerical_input = tf.keras.layers.Input(...)
circuit_input = tf.keras.layers.Input(...)
a = tf.layers.Dense(2)(numerical_input)

tf_learnable_gamma = tf.Variable(...)

datapoint_circuit_plus_model_circuit = tfq.layers.AddCircuit()(circuit_input, quantum_model_circuit)
b = tfq.layers.Expectation()(
  datapoint_circuit_plus_model_circuit,
  symbol_names = datapoint_symbols + model_symbols,
  symbol_values = tf.concat([a, tf_learnable_gamma], axis=1),
  operators=Some operator)
c = tf.layers.Dense(5)(b)
.
.
.
out = ...
model = tf.keras.Model(inputs=[numerical_input, circuit_input], outputs=out)
model([some tensor of numbers, datapoint_circuits])
```

Here you can see that the gradients of alpha and beta will backprop through `a` whereas the gradients of gamma will backprop into `tf_learnable_gamma` which are the tf learnable parameters for your `quantum_model_circuit`. This lets `a` and any other modelling upstream from `a` have an influence over the structure of the quantum datapoint circuits going into the quantum model. Is this what you are after ?

1

MichaelBroughton self-assigned this on Jul 31, 2020

m0tela01 on Jul 31, 2020 — Author

@MichaelBroughton Thank you for the response, hope your vacation is going well.

You are correct that the first bit of code is not what I am after. I believe it is the second set of code that is correct. I have tried to use `tfq.layers.Expectation` already but the way that you show it above at least appears to be what I want (will implement and let you know). In the meantime I have a question about the two circuits `quantum_model_circuit` and `datapoint_circuits` . Is `datapoint_circuits` the dataset (like MNIST) that is already converted to quantum states, if so in the code above are you feeding both the classical and quantum versions of the data? If that is the case then I believe my only other questions would be (1) could you explain why/if its OK to pass both the classical and quantum versions of the data as the input (2) what exactly is `tfq.layers.Expectation` doing to the tensor? For (1) I initially rejected the idea of trying this because it did not make sense to me to give the model both versions of the data and (2) it appears `tfq.layers.Expectation` is much more powerful than just using a PQC/ControlledPQC.

Thank you for coded-explanation its very helpful.

MichaelBroughton on Aug 3, 2020 — Collaborator

Glad I was able to help clear things up :)

> could you explain why/if its OK to pass both the classical and quantum versions of the data as the input

You can pass whatever you want into tensorflow compute graphs as inputs. It's OK to pass associated classical information to go along with your quantum circuits. I would just be careful to not accidentally do all the learning on a classical only model that just learns to ignore the harder to learn quantum parts.

> what exactly is tfq.layers.Expectation doing to the tensor?

I would check out this: https://www.tensorflow.org/quantum/api_docs/python/tfq/layers/Expectation which goes over how `tfq.layers.Expectation` works and the types of tensor input formats it can accept. `tfq.layers.Expectation` is a little more low level than a `PQC` / `ControlledPQC` layer so in the case where you need a little more fine grained control it can help out a lot.

👍 1

m0tela01 on Aug 5, 2020 · edited by m0tela01 — Edits — Author

@MichaelBroughton
It appears this is the correct solution but I have an error when implementing this.

Error:

```
InvalidArgumentError: 2 root error(s) found.
  (0) Invalid argument:  Incompatible shapes: [1,16,8] vs. [1,16,47]
        [[{{node PartitionedCall/map/while/body/_15/Add_1}}]]
  (1) Invalid argument:  Incompatible shapes: [1,16,8] vs. [1,16,47]
        [[{{node PartitionedCall/map/while/body/_15/Add_1}}]]
        [[PartitionedCall/Reshape_1/_82]]
0 successful operations.
0 derived errors ignored. [Op:__inference_distributed_function_1552]

Function call stack:
distributed_function -> distributed_function
```

I got a very similar error when I had the wrong shape for `learnable_gamma` where the error is pretty much the same but the incompatible shapes are `[4,1,32] vs. [16,1,32]` . Not sure what shape is related to the shapes `[1,16,8] vs. [1,16,47]` in the error above. I noticed changing the batch size does influence the shapes shown in the error but it never fixes the issue. I found that if I changed the `dense_in` below to 43 units the model will train 1 epoch but then fail with the same error but with the shapes `[16,1,32] vs. [14,1,32]` .

The batch size I am using is 16

Code:

```
data_syms = sympy.symbols("a b c d e")
qcnn_syms = sympy.symbols('qconv0:42')


bit = cirq.GridQubit(0, 0)
ops = [-1.0 * cirq.Z(bit), cirq.X(bit) + 2.0 * cirq.Z(bit)]
learnable_gamma = tf.Variable([[1.,1.,1.,1.]]*16, dtype=float)
cluster_state_bits = cirq.GridQubit.rect(1, 8)
quantum_cnn = qcnn.multi_readout_model_circuit(cluster_state_bits, qcnn_syms)


dense_in = tf.keras.layers.Dense(4)(x)

datapoint_circuit_plus_model_circuit = tfq.layers.AddCircuit()(circuit_input, prepend=quantum_cnn)
quantum_expectation = tfq.layers.Expectation()(
  datapoint_circuit_plus_model_circuit,
  symbol_names = data_syms + qcnn_syms,
  symbol_values = tf.concat([dense_in, learnable_gamma], axis=1),
  operators=ops,)

d2_dual = tf.keras.layers.Dense(1)(quantum_expectation)
```

Architecture:

```
Model: "model_2"
__________________________________________________________________________________________________
Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
input_5 (InputLayer)            [(None, 4, 1)]       0
__________________________________________________________________________________________________
conv1d_2 (Conv1D)               (None, 3, 32)        96          input_5[0][0]
__________________________________________________________________________________________________
max_pooling1d_2 (MaxPooling1D)  (None, 1, 32)        0           conv1d_2[0][0]
__________________________________________________________________________________________________
dropout_2 (Dropout)             (None, 1, 32)        0           max_pooling1d_2[0][0]
__________________________________________________________________________________________________
flatten_2 (Flatten)             (None, 32)           0           dropout_2[0][0]
__________________________________________________________________________________________________
input_6 (InputLayer)            [(None,)]            0
__________________________________________________________________________________________________
dense_6 (Dense)                 (None, 4)            132         flatten_2[0][0]
__________________________________________________________________________________________________
add_circuit_2 (AddCircuit)      (None,)              0           input_6[0][0]
__________________________________________________________________________________________________
tf_op_layer_concat_2 (TensorFlo [(16, 8)]            0           dense_6[0][0]
__________________________________________________________________________________________________
expectation_2 (Expectation)     (None, 2)            0           add_circuit_2[0][0]
__________________________________________________________________________________________________
dense_8 (Dense)                 (None, 1)            3           expectation_2[0][0]
==================================================================================================
Total params: 231
Trainable params: 231
Non-trainable params: 0
```

MichaelBroughton on Aug 5, 2020 — Collaborator

This seems to be a good old fashioned shape error now. Your snippet doesn't contain enough code for me to diagnose what is happening (where does `x` come from in `tf.keras.layers.Dense(4)(x)` ?). I would recommend slowly building up your model, printing out the shapes as you go, keeping a very close eye on how ops change behave with certain shape inputs, so that you get the expected shapes. We are moving pretty far from the original issue of upstream variable usage in quantum operations.

If it manages to work for a whole epoch, but then not the next one, you might want to make sure that the data you are sending in for each epoch has the same batch size.

Think we are good to close now :)

m0tela01 closed this as completed on Aug 5, 2020

m0tela01 on Aug 5, 2020 Author ···

@MichaelBroughton
I've closed the issue but was not sure if I should ask this question here or open another issue..

From above you can simplify the architecture so that `x=tf.keras.Input(shape=(4,))` and that would be all of the relevant code. What I believe the issue may be is the lack of some kind of **readout** parameter for `tfq.layers.Expecation` in order for the shape to output from either the `tfq.layers.AddCircuit()` or `tfq.layers.Expectation` correctly. I do not see readouts for AddCircuit or Expectation in the documentation.

I've narrowed the the issue to what I believe is the concatenation `tf.concat([dense_in, learnable_gamma], axis=1),` for the symbol_values... was hoping this might be of some help in explaining this. I spent the better part of yesterday and today fiddling with shapes of tensors already, changing/reshaping tensors does not really seem like the issue.

Not sure what you meant by:

> make sure that the data you are sending in for each epoch has the same batch size

Was under the impression sending data using Keras like below guaranteed that batches were uniform

```
history = hybrid_model.fit(x=[x_train,x_train_q],
                           y=y_train1,
                           batch_size=16,
                           epochs=25)
```

For completeness the whole code:

```
data_syms = sympy.symbols("a b c d e")
qcnn_syms = sympy.symbols('qconv0:42')


bit = cirq.GridQubit(0, 0)
ops = [-1.0 * cirq.Z(bit), cirq.X(bit) + 2.0 * cirq.Z(bit)]
learnable_gamma = tf.Variable([[1.,1.,1.,1.]]*16, dtype=float)
cluster_state_bits = cirq.GridQubit.rect(1, 8)
quantum_cnn = qcnn.multi_readout_model_circuit(cluster_state_bits, qcnn_syms)


input_shape = tf.keras.Input(shape=(4,))
circuit_input = tf.keras.Input(shape=(), dtype=tf.dtypes.string)
dense_in = tf.keras.layers.Dense(4)(input_shape)        ### units=43 will train for 1 epoch

datapoint_circuit_plus_model_circuit = tfq.layers.AddCircuit()(circuit_input, prepend=quantum_cnn)
quantum_expectation = tfq.layers.Expectation()(
  datapoint_circuit_plus_model_circuit,
  symbol_names = data_syms + qcnn_syms,
  symbol_values = tf.concat([dense_in, learnable_gamma], axis=1),
  operators=ops,)

d2_dual = tf.keras.layers.Dense(1)(quantum_expectation)

hybrid_model = tf.keras.Model(inputs=[input_shape, circuit_input], outputs=d2_dual)

hybrid_model2.compile(tf.keras.optimizers.Adam(learning_rate=0.02),
                    loss=tf.keras.losses.mse,
                    metrics=[h.custom_accuracy])

history = hybrid_model.fit(x=[x_train,x_train_q],
                           y=y_train1,
                           batch_size=16,
                           epochs=25,
                           validation_data=([x_val, x_val_q], y_val1))
```

m0tela01 on Aug 6, 2020 — Author

@MichaelBroughton
I am reopening the issue because the solution seems to either have a bug or be incomplete (I am following exactly what you have provided above). I don't believe this is a separate issue related to shapes. It appears the learnable parameters in the expectation layer are not able to be learned?

add_circuit was passed non-serializable keyword arguments

Error:

```
WARNING:tensorflow:Layer add_circuit was passed non-serializable keyword arguments: {'append': cirq.Circuit([
    cirq.Moment(
        (cirq.X**sympy.Symbol('qconv0')).on(cirq.GridQubit(0, 0)),
    ),
....
....
    cirq.Moment(
        (cirq.X**sympy.Mul(sympy.Integer(-1), sympy.Symbol('qconv36'))).on(cirq.GridQubit(0, 7)),
    ),
])}. They will not be included in the serialized model (and thus will be missing at deserialization time).
WARNING:tensorflow:Layer expectation was passed non-serializable keyword arguments: {'symbol_names': (a, b, c,
qconv1, qconv2, qconv3, qconv4, qconv5, qconv6, qconv7, qconv8, qconv9, qconv10, qconv11, qconv12, qconv13, qc
qconv15, qconv16, qconv17, qconv18, qconv19, qconv20, qconv21, qconv22, qconv23, qconv24, qconv25, qconv26, qc
qconv28, qconv29, qconv30, qconv31, qconv32, qconv33, qconv34, qconv35, qconv36, qconv37, qconv38, qconv39, qc
qconv41), 'symbol_values': , 'operators': [-cirq.Z(cirq.GridQubit(0, 0)),
cirq.PauliSum(cirq.LinearDict({frozenset({(cirq.GridQubit(0, 0), cirq.X)}): (1+0j), frozenset({(cirq.GridQubit
 They will not be included in the serialized model (and thus will be missing at deserialization time).
WARNING:tensorflow:Model failed to serialize as JSON. Ignoring... Layers with arguments in `__init__` must ove
```

zaqqwerty on Aug 6, 2020 · edited by zaqqwerty — Edits — Contributor

This doesn't look like an issue with training, this looks like an issue with model saving (see the `get_config` call out at the end of the warning message.) This is noted in issue #56 and #275. Since what you added above are warnings and not errors, were there subsequent errors that caused the training not to proceed?

m0tela01 on Aug 6, 2020 · edited by m0tela01 — Edits — Author

@zaqqwerty I was trying to use tensorboard's graph to find where my shapes may have an issue. This error is shown above the previous error when trying to train:

Previous/Current Error:

> InvalidArgumentError: 2 root error(s) found.
> (0) Invalid argument: Incompatible shapes: [1,16,8] vs. [1,16,47]
> [[{{node PartitionedCall/map/while/body/_15/Add_1}}]]
> (1) Invalid argument: Incompatible shapes: [1,16,8] vs. [1,16,47]
> [[{{node PartitionedCall/map/while/body/_15/Add_1}}]]
> [[PartitionedCall/Reshape_1/_82]]
> 0 successful operations.
> 0 derived errors ignored. [Op:__inference_distributed_function_1552]
> Function call stack:
> distributed_function -> distributed_function

You are correct, this most recent snippet is related to saving. The real issue I am having is related to `learnable_gama`, `AddCircuit()` and the expectation layer. Tensorflow will not train this layer as per the example @MichaelBroughton provided. Fixing the shapes does not appear to be the solution. The shape values in the error changes with a change to `batch_size`, units in dense layers, the size of `learnable_gamma`. I can't seem to sort out a specific part of the network that has the wrong shape. When the shapes are "correct" the model trains the 0th epoch and comes back to the same error. When the shapes are "wrong" the model shows the same error with different shapes.

@MichaelBroughton @zaqqwerty

I have created a dummy notebook replicating my error using the implementation for `tfq.layers.Expectation` provided in this issue and the MNIST notebook (simplified) provided in your TFQ tutorials.

Please take a look here:
https://colab.research.google.com/drive/12x1ELFr5OKdL5sHq8Wea3cre_FzHvpN8?usp=sharing

The dense layer before the expectation layer here can be changed to 30 units and the model will run for one epoch. This is a link to that cell.
https://colab.research.google.com/drive/12x1ELFr5OKdL5sHq8Wea3cre_FzHvpN8?authuser=1#scrollTo=-Gmm0aA8TL5g&line=10&uniqifier=1

MichaelBroughton on Aug 6, 2020 — Collaborator

Ok, turns out that the error was indeed a good old fashioned shape mismatch. I added a little snippet in the notebook to show where:

```
print('Required number of parameters for exp layer:', (None, len(data_syms + model_syms)))
print('Provided number of parameters for exp layer:', tf.concat([dense_in, learnable_gamma], axis=1).sha
print('Should be equal on 2nd dim.')
```

Output of the above is `(None, 34)` and `(8,12)` (or something). The 2nd dim's need to line up so that every free symbol has exactly one value to be placed inside of it.

Does this clear things up now ?

👎 1

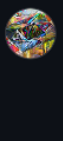
m0tela01 on Aug 6, 2020 — Author

@MichaelBroughton
Did you change the units in the Dense layer to 30 like I mentioned?
I knew the shapes were wrong when I sent it to you, I wanted you to see both errors. The second dimension will be correct if you use 30 units in `dense_in`

> The dense layer before the expectation layer here can be changed to 30 units and the model will run for one epoch. This is a link to that cell.
> https://colab.research.google.com/drive/12x1ELFr5OKdL5sHq8Wea3cre_FzHvpN8?authuser=1#scrollTo=-Gmm0aA8TL5g&line=10&uniqifier=1

I changed `dense_in` 30 units for you so that the shapes of the second dimension are correct. The model only trained for one epoch again.

Please take a look now



MichaelBroughton on Aug 7, 2020 · edited by MichaelBroughton — Edits — Collaborator

Sorry if I wasn't clear in the first reply. You need to make sure the shapes line up in the 2nd dimension. On top of that you also need to make sure that you aren't using static sized shapes in your model. `(None, 34) != (4, 34)`, even if one sets `batch_size=4` this seeming equality should not be relied upon. The reason your code is not working is because you have something like 22 examples in your training data. With a batch size of 4 your model needs to work when `batch_size = 4` and on the last datapoint in the batch where the `batch_size= 2`. I've gone ahead and made a working copy of the notebook that will train: https://colab.research.google.com/drive/1Ve5i6OuqJZxc6jAumEtPwUUZPe_va9Zx?usp=sharing . All I really did was set `batch_size=2`. The issue of fixing the static shaping problem so that you can make your `batch_size` something other than 2, I will leave to you.

In order to make your model more robust I would recommend that you instead of introducing static sized variables at your batch size, tile them up appropriately so that you don't get things like `(4 , X)` and `(None, X)` which are bound to have shape problems.